AOSP 进阶丛书 · 第十册 · ANDROID 13 (API 33)

网络栈源码全解

Connectivity 模块化新布局、网络评分与验证四探针、netd 与 eBPF 流量记账、DNS 与 Private DNS 三态、VPN tun 机制与 Tethering offload——一次请求要过的全部系统关卡。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：网络是一组"被治理的路径" 2
第一章 全链路：一次请求要过几道关 2
第二章 网络请求与评分：怎么选默认网 3
第三章 验证：这网能不能信 4
第四章 netd 与 eBPF：流量从记账到限速 5
第五章 DNS：Private DNS 三态 6
第六章 VPN：tun 设备与路由劫持 6
第七章 设计决策八问 7
附录 A 观测实操 8
附录 B 快问快答（16 条） 8
附录 C 自检题 9

本册速览

模块化新布局：Connectivity→packages/modules/Connectivity（Mainline）
评分与验证：NetworkRanker 挑默认网 · NetworkMonitor 四探针
netd 与 eBPF：tagSocket→uid_tag_map→流量账本 · Tethering offload
DNS 三态与 VPN：Private DNS(DoT) · tun 设备与 protect 防环
附录 A 观测表 + 第六章设计决策八问

前言：网络是一组"被治理的路径"

对应用开发者网络是"默认有网"；对系统开发者网络是**一组被策略治理的路径**：哪个网络可用（验证）、谁优先（评分）、谁的流量走哪（路由）、流量记到谁头上（eBPF 记账）、DNS 是否加密（Private DNS）、全量代理怎么不绕圈（VPN protect）。本册沿这六问展开。

路径新布局（读码第一坑）：本树（Android 13）里 Connectivity 已 Mainline 化——`packages/modules/Connectivity/`（记 CS=…/service/src/com/android/server）；NetworkMonitor 在**独立的 NetworkStack 模块**（packages/modules/NetworkStack/）；netd 在 system/netd/server/（main.cpp:111）；而 Vpn.java 仍在 frameworks/base/services/core/…/connectivity/（:179）。旧教程的 frameworks/base/services/.../connectivity 路径在本树**只余 Vpn.java 一个活化石**。

第一章 全链路：一次请求要过几道关



图 1 网络栈全链路：连接管理（CS）→验证（NetworkMonitor）→数据落地（netd/eBPF）三段，含模块化迁移对照表

三段分工一句话：**CS 管决策（谁当默认网）、NetworkMonitor 管验证（这网能不能信）、netd 管落地（DNS/规则/记账）**。App 侧的每个 API（ConnectivityManager/TrafficStats/私有 DNS 设置）都能在这三段里找到实体。

第二章 网络请求与评分：怎么选默认网



图 2 请求-评分时序：NetworkRequest→NetworkFactory→NetworkAgentInfo→FullScore→Ranker→linger 切换

2.1 逐段注释（真实锚点）

java packages/modules/Connectivity/…（骨架+讲解注释）

```
// ConnectivityService.java:314（11435 行！Mainline 后的巨石仍在这里）
public class ConnectivityService extends IConnectivityManager.Stub {
// 讲解注释①：requestNetwork(:1678 createNetworkRequest)：
// App 声明"我要什么"（能力：INTERNET+NOT_METERED+…）；
// 供给侧 NetworkFactory（wifi/cell/eth 各自一个）声明"我有什么"。
// 讲解注释②：NetworkAgentInfo（connectivity/NetworkAgentInfo.java）：
// 一个"活网络"的运行时视图：agent+FullScore+linkProperties+验证状态。
// 每次网络状态变化，Nai 更新并触发 rematch。
// 讲解注释③：NetworkRanker（connectivity/NetworkRanker.java）：
// 对每个请求，在满足其能力的网络里选 FullScore 最高者。
// 讲解注释④：切换的缓冲——LingerMonitor：老网进入 linger 宽限期
// （默认 30s），期间 App 的 bindSocket 还连着老网；宽限后真断。
// "Wi-Fi 闪断瞬间 App 没断"的秘密。
}
```

设计洞察 · 逐段精读：为什么 CS 有一万一千行

它同时是：请求注册中心（本节）、网络评分裁判（Ranker）、Airplane 模式、代理（ProxyTracker）、Keepalive（TcpKeepaliveController）、多网并发（profile preference）……Mainline 化后这团复杂度可独立升级（com.android.tethering APEX），但**没有拆小**——网络状态的全局一致性比代码切分更稀缺。

2.3 多网络并发与 profile（企业/多场景）

13 的 ProfileNetworkPreferenceList（CS/connectivity/ 目录）：按用户 profile 指定"办公流量走企业 APN、其他走默认"——**同一进程内两个网络身份并存**。App 侧 bindSocketToNetwork 显式选网（多开下载器的场景）。读码入口 `ConnectivityService#setProfileNetworkPreference`；与 per-app VPN（第六章）是两种"分流量"手段（网络级 vs 隧道级）。

2.2 观测与排障

`dumpsys connectivity`（NetworkAgentInfo 全表：分数/验证/linger）；`dumpsys connectivity airplane`（飞行模式）；网切日志：logcat ConnectivityService 标签的 rematch 记录。经典题："Wi-Fi 与蜂窝同时开为什么走 Wi-Fi"——TRUSTED+NOT_METERED 加权后 FullScore 稳赢。

第三章 验证：这网能不能信



图 3 NetworkMonitor 状态机与四探针（:225 StateMachine、:29-33 PROBE 常量），强制门户与部分连通的下游影响

`NetworkMonitor`（NetworkStack/…/NetworkMonitor.java:225，extends StateMachine——frameworks 的"小 Handler 状态机"库）对每个网络跑四探针：PROBE_DNS/HTTP/HTTPS/PRIVDNS（:29-33）。判例集：302 重定向→CAPTIVE_PORTAL（弹登录页）；HTTPS 不通 HTTP 通→PARTIAL_CONNECTIVITY（"无法访问互联网"提示但 fallback 可用）。验证**异步**：先给网后验网，结论回来重新评分。面试高频："连上 Wi-Fi 提示无法上网但仍能用"= PARTIAL 语义。

第四章 netd 与 eBPF：流量从记账到限速



图 4 eBPF 流量记账链：setThreadStatsTag→tagSocket→cookie map→uid 计数→读取，及 Tethering offload

4.1 数据链逐段

text system/netd/server（结构+讲解注释）

```
main.cpp:111 int main()
// 讲解注释①：netd 承担 CS 的 native 执行：DNS resolver、
// 带宽/防火墙规则（BandwidthController.cpp 同目录）、
// socket tag（BpfNetMaps.java:37 是 Java 门面，native 在 netd）。
// 讲解注释②：eBPF 记账：
// App: TrafficStats.setThreadStatsTag(0xF00D)
// → 新 socket 创建时 tagSocket：内核 eBPF 程序按 socket cookie
// 写 uid_tag_map（uid+tag ↔ cookie）
// → 每包经过：cgroup hook 查 cookie → 累加到 per-uid-tag 计数 map
// → TrafficStats.getUidRxBytes → binder → 读 map 快照
// 讲解注释③：iptables 只剩隔离（UID 禁网），记账全 eBPF：
// map 查 O(1)、可按任意维度聚合——这是 Settings 流量页的后端。
```

设计洞察 · 逐段精读：tag 为什么是 int

TrafficStats 的 tag 是 32 位（应用可自定高 16 位做子类统计，低 16 位是 uid 校验）——同一 App 内区分"图片流量/广告流量"的正统做法。面试追问"防作弊吗"——不防：tag 是 App 自报的，但 uid 部分内核核对（伪造 tag 逃不掉 uid 账）。

4.2 Tethering 的 BPF offload

`IpServer`（Tethering/…/IpServer.java）：每个热点下游接口一个实例（DHCP/NAT/限速）；**BpfCoordinator** 做下行 offload——包直走硬件转发不上 CPU（省电主战场）。观测 `dumpsys connectivity tether`。

第五章 DNS：Private DNS 三态



图 5 DNS 解析链：bionic→dnsproxyd→netd resolver，DoT 三态与排障三题

三态（DnsManager.java:24-26 常量）：OFF（明文 53）/STRICT（DoT：TLS 853 到指定主机，失败即断不解）/AUTO（先试 TLS 失败回落）。**App 不直接发 DNS**——bionic getaddrinfo 走 `/dev/socket/dnsproxyd` 到 netd：多网络按调用方绑定网选 DNS 服务器，只有系统知道该用哪张网。排障 `dumpsys dnsresolver`；STRICT 配错主机=整体解析失败（不回落）。

第六章 VPN：tun 设备与路由劫持



图 6 VPN 机制：establish(:1504) 建 tun→路由劫持→protect 防环，平台 VPN 与 always-on/lockdown

text frameworks/.../connectivity/Vpn.java:179/1504（结构+讲解注释）

```
// 讲解注释①：establish(:1504)：TUNSETIFF 建 tun0，返回
// ParcelFileDescriptor ——fd 跨进程（书3 BINDER_TYPE_FDA）
// 讲解注释②：路由劫持：addRoute 的 CIDR 写内核路由（默认 0/0 全量）
// 讲解注释③：protect()：加密通道 socket 标记"绕开 tun"——防递归环
//  （VPN 流量再进 tun=自己套自己）
// 平台 VPN：VpnProfile/IkeSessionParams——密钥协商框架管，App 只拿 fd
// always-on/lockdown：开机自启/断 VPN 即断网（不回落明文）
```

数据面一句话：**VPN App 是用户态路由器**（read tun→加密→protect socket 发物理网→回程 write tun），性能瓶颈在加解密与两次上下文切换。

第七章 设计决策八问

- **为什么网络全 Mainline 化？** 运营商/安全补丁需要急速分发（拖整机 OTA 等不起）；接口（NetworkAgent AIDL）稳定是前提。
- **为什么验证异步不阻塞？** 可用性优先：部分连通先顶上，结论回来再调整——比"等验完才给网"体验好。
- **为什么 DNS 走 socket 不走 binder？** bionic 是 C 库（无 binder C API）；且 fork 出的进程禁 binder（书2 §六 呼应）。
- **为什么记账用 eBPF？** O(1) map 查询+任意维度聚合；iptables 计数 O(rules) 且写规则慢。
- **为什么 STRICT 失败不回落？** 用户明示"只要加密"——回落即违背意愿（安全语义 > 可用性）。
- **为什么 VPN protect 必须显式？** 隐式判断"哪些 socket 是隧道"不可靠（框架无法识别加密流量的目的）；责任交给最知情的 VPN App。

- **为什么 linger 不立即断老网？** 切换抖动（Wi-Fi 闪断）频繁重连代价高；宽限期吸收毛刺。
- **为什么 NetworkMonitor 独立成 NetworkStack 模块？** 验证逻辑与 captive portal 协议高频演进；且被多模块复用（Wi-Fi/CS/Tethering 都要问"这网通吗"）。

附录 A 观测实操

目标	命令	看点	
网络全表	dumpsys connectivity	NetworkAgentInfo：分数/验证/linger	
验证历史	dumpsys connectivity |	grep -A5 Validated	探针结论
DNS	dumpsys dnsresolver	缓存命中/服务器/Private DNS 模式	
eBPF/流量	dumpsys netd	tag map/规则	
热点	dumpsys connectivity tether	下游客户端/offload	
VPN	dumpsys connectivity |	grep -A5 VPN	tun/uid 范围/always-on
路由	ip route show table all	tun/多网路由表	

附录 B 快问快答（16 条）

- 本树 CS 在哪？——packages/modules/Connectivity/（Mainline）。
- NetworkMonitor 在哪？——packages/modules/NetworkStack/（独立模块）。
- Vpn.java 为何没走？——frameworks/base/services/core/.../connectivity/Vpn.java:179（活化石）。
- 四探针是哪四个？——DNS/HTTP/HTTPS/PRIVDNS（:29-33）。
- PARTIAL 什么语义？——HTTPS 不通但 HTTP 通：fallback 可用。
- 默认网怎么选？——Ranker 在满足能力的网里挑 FullScore 最高。
- linger 多久？——默认 30s（吸收切换毛刺）。
- tagSocket 写到哪个 map？——uid_tag_map（cookie→uid+tag）。
- TrafficStats tag 谁核对？——uid 部分内核核对，tag 高 16 位 App 自由。
- iptables 现在管什么？——UID 隔离/限速；记账已 eBPF。
- Private DNS STRICT 失败？——整体解析失败（不回落明文）。
- DoH 系统支持吗？——默认 DoT（853）；DoH 是 App 库层行为。
- VPN 防环靠什么？——protect() 标记隧道 socket 绕 tun。

- always-on 与 lockdown 区别？——自启重连 vs 无 VPN 即断网。
- Tethering offload 价值？——下行硬件转发不上 CPU（省电）。
- 学完往哪走？——书11 存储加密（data 完整性的另一半）。

附录 C 自检题

- 默画"请求→评分→默认网→linger"时序；Wi-Fi/蜂窝分差翻转时 App 感知什么。
- 四探针各测什么？302 与 HTTPS 超时分别判什么？
- 一个 socket 的流量从 tag 到被 Settings 统计的全链（含 map 名）。
- Private DNS 三态与失败语义；为什么 DNS 必须过 netd。
- VPN 数据面一个包的双程；protect 缺失会发生什么。